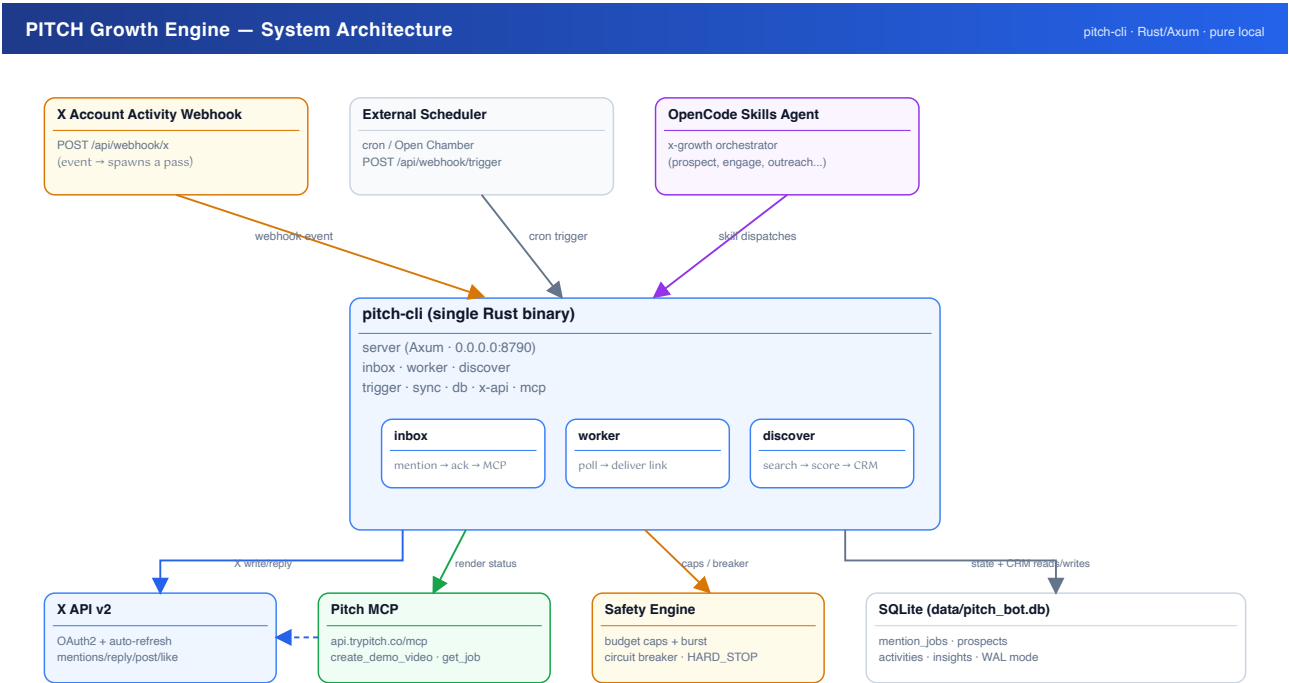


PITCHXGrowthEngine

Developer Documentation & Execution Guide

Webhooks · Triggered Skills · Business Accomplishments (ROI) · Schedules & Cron · Setup



Real-Time Webhook X
mentions → instant receipt →
AI video render → delivery re-
ply in-thread.

Cron & Polling Scheduled
passes recover lost events, ad-
vance renders, and discover
SaaS leads.

Safety Enforced All write actions gated by budget caps, burst limits, and circuit breaker.

Table of Contents

Contents

1	Introduction & System Overview	4
2	The CLI Surface	4
3	Webhook Endpoints → Skill Mapping → Business Accomplishments	6
3.1	Endpoint & Skill Objective Breakdown	6
4	Request Flows	8
4.1	Flow A — Real-time mention → demo delivery (x-mention skill)	8
4.2	Flow B — Manual trigger endpoint	8
4.3	Flow C — Outbound prospect discovery (x-prospect skill)	9
4.4	Flow D — Health, stats & observability	9
5	Webhook Routes Table	10
6	Job & Prospect State Machines	10
7	Database Schema & State Mirroring	11
8	Schedules & Cron	11
8.1	The recovery net (why you still need cron)	11
8.2	Server (always-on)	12
8.3	crontab	12
8.4	Schedule summary	12
9	Safety Engine	13
9.1	Circuit breaker	13
9.2	Budget caps (defaults, scaled ×0.25 during cold-start ramp)	13
10	X API v2 & Pitch MCP	14
10.1	X API v2	14
10.2	Pitch MCP	14
11	Setup & Environment	15
11.1	Requirements	15
11.2	.env (gitignored — see src/config.rs)	15
11.3	Build & verify	15
11.4	Wire the webhook in X	15
12	Automation Checklist	15

1 Introduction & System Overview

The *PITCH X Growth Engine* runs the X/Twitter growth-to-sales funnel for <https://trypitch.co> — an AI video editor that turns task descriptions into narrated demo MP4s. This engine watches mentions of @trypitchdotco, renders a product demo for any URL a user posts, replies in-thread with the finished video, and runs outbound prospect discovery. Everything runs from a single compiled Rust binary (`pitch-cli`) plus a local SQLite database.

Two mechanisms drive it:

1. *Real-time webhook* — X calls `POST /api/webhook/x` on every new mention. The server immediately spawns a background pass (`inbox` then `worker`) and returns 200.
2. *Scheduled passes* — cron invokes CLI subcommands (`trigger`, `inbox`, `worker`, `discover`) or OpenCode skills so the pipeline keeps running even if a webhook event is lost. The SQLite job table makes every pass idempotent.

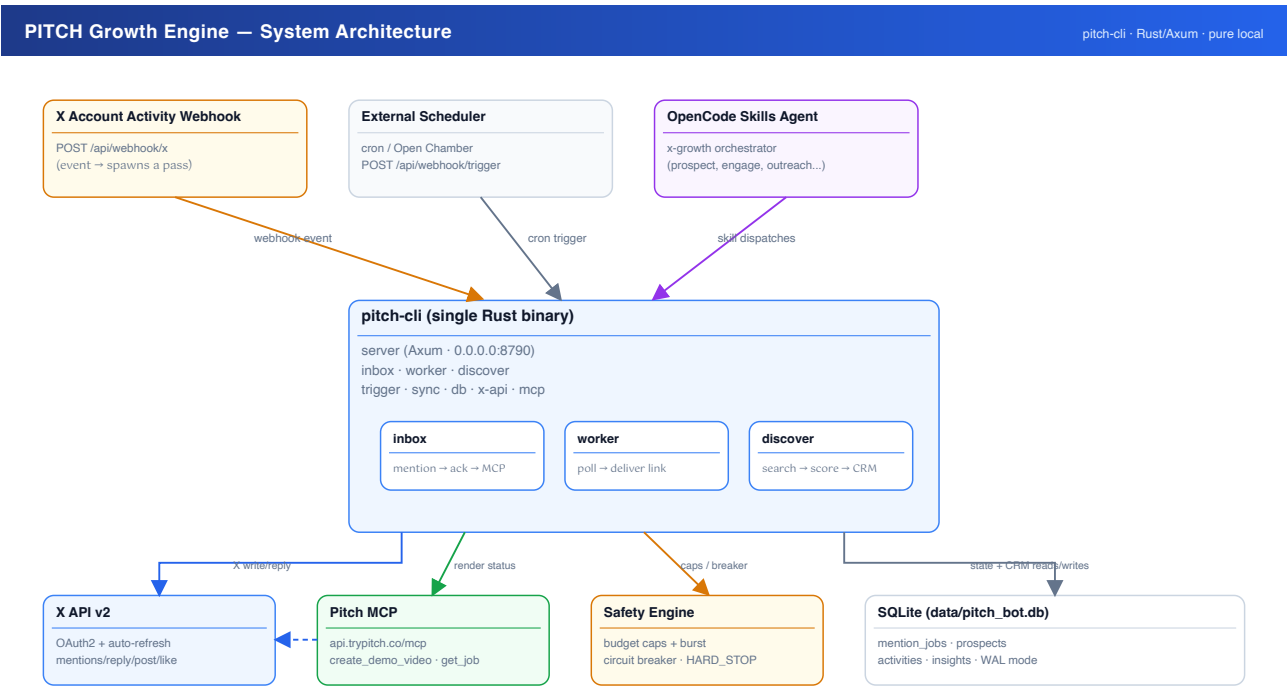


Figure 1: System architecture — single binary, local SQLite, two trigger paths.

Note *One binary, one database, zero external middleware.* The webhook server, X API client, Pitch MCP client, safety engine, and SQLite are all in `src/`. Data lives in `data/pitch_bot.db` (WAL mode) unless `SQLITE_DB_PATH` overrides it.

2 The CLI Surface

`pitch-cli` is a clap-based CLI. Every operation is a subcommand; most pipeline commands accept `--dry` to preview behavior without writing to X or Pitch MCP.

Command	Purpose
<code>pitch-cli server [--port N], Embedded Axum server bound to 0.0.0.0:8790 (default) — hosts all webhook routes</code>	<code>pitch-cli inbox [--dry] [--no-ack]</code> , Ingest recent mentions, archive / dedupe, ack, submit Pitch MCP render jobs

pitch-cli worker [--dry] [--limit N], Poll Pitch MCP for job status; deliver S3 video links as X replies	pitch-cli discover [--dry] [--max N], Run the ICP search queries, score leads, upsert into the CRM
pitch-cli trigger [--dry] [--no-ack], One-shot combined pass: inbox + worker + DB summary (the cron job)	pitch-cli sync, Print a summary of mention jobs and prospects in the DB
pitch-cli db jobs prospects insights get, Query SQLite (jobs by status, CRM prospects, adaptive memory)	pitch-cli x-api me refresh mentions reply post like search, X API v2 operations with OAuth2 auto-refresh on 401
pitch-cli mcp create status credits, Pitch MCP calls, e.g. create <url>, status <job_id>, credits	pitch-cli budget, Daily action caps + rolling 1-hour burst usage
pitch-cli circuit-breaker [--trip "reason"] [--reset], Query, trip, or reset the kill-switch	

3 Webhook Endpoints → Skill Mapping → Business Accomplishments

The system exposes 5 HTTP webhook routes via Axum. Below is the full mapping showing how each exposed webhook endpoint triggers a specific skill, executes a pipeline routine, and produces a concrete business accomplishment (ROI) for PITCH:

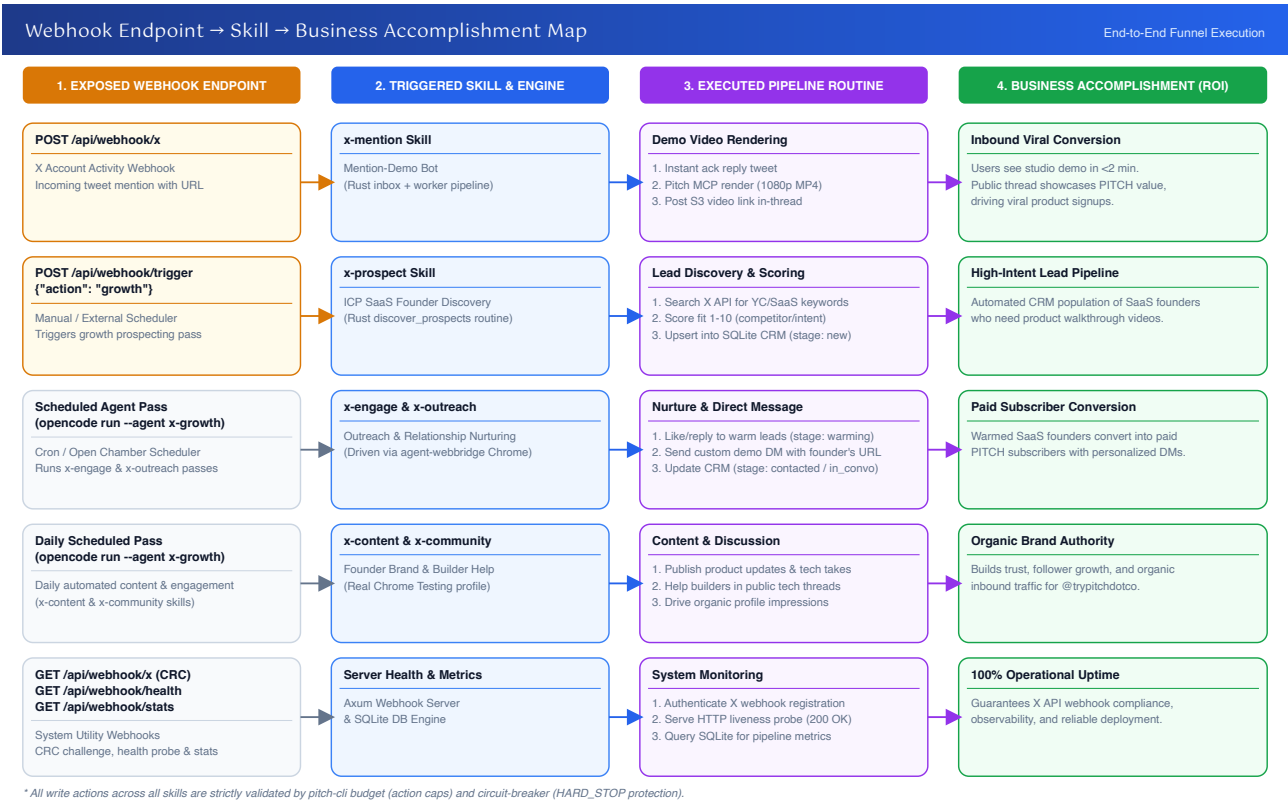


Figure 2: Complete Mapping: Exposed Webhook Endpoint → Triggered Skill → Pipeline Routine → Business Accomplishment.

3.1 Endpoint & Skill Objective Breakdown

Exposed End-point	Triggered Skill	Pipeline Routine	Business Accomplishment (ROI)
POST /api/webhook/x, x-mention, 1. Ack reply tweet 2. Pitch MCP 1080p render 3. S3 video re- ply in-thread, In- bound Viral Demo Funnel: Users re- ceive studio-qual- ity video demo of their URL in un- der 2 min. Public replies showcase PITCH value, dri- ving viral signups.	POST /api/webhook/trigger {"action": "growth"} x-prospect, 1. Search X API v2 for SaaS key- words 2. Lead fit score (1-10) 3. Upsert to SQLite (stage: new), High-Intent Lead Pipeline: Auto- matically popu- lates CRM with SaaS founders who need video walkthroughs for launches.	POST /api/webhook/trigger {"action": "mentions"}, x-mention, 1. Fetch unhandled mentions 2. Submit pending renders to MCP 3. Deliver finished video replies, Inbound Recovery Net: Guarantees zero missed mentions even if an X webhook callback drops.	Scheduled Agent Pass opencode run --agent x-growth, x-engage x-outreach, 1. Like/reply warm leads (stage: warming) 2. Custom DM with founder's URL demo 3. Update CRM (stage: contacted), Paid Subscriber Conversion: Nurtures SaaS founders in public, then converts them in DMs into paid subscribers.

Daily Scheduled Pass <code>opencode run --agent x-growth, x-content x-community, 1.</code> Post product updates & tech takes 2. Help builders in public threads 3. Drive organic profile impressions, <i>Brand Authority & Awareness</i>: Builds trust, follower growth, and organic inbound traffic for @trypitchdotco.	GET <code>/api/webhook/x,</code> System Utility, HMAC-SHA256 CRC token verification challenge, <i>X API Compliance</i> : Required by X Account Activity API to register webhooks.	GET <code>/api/webhook/health,</code> System Utility, Liveness probe returning HTTP 200 OK, <i>Deployment Health</i> : Ensures webhook server is operational.	GET <code>/api/webhook/stats,</code> System Utility, Pipeline metrics query from SQLite, <i>Observability</i> : Monitors total jobs, renders, and CRM prospects.
--	---	---	--

4 Request Flows

4.1 Flow A — Real-time mention → demo delivery (x-mention skill)

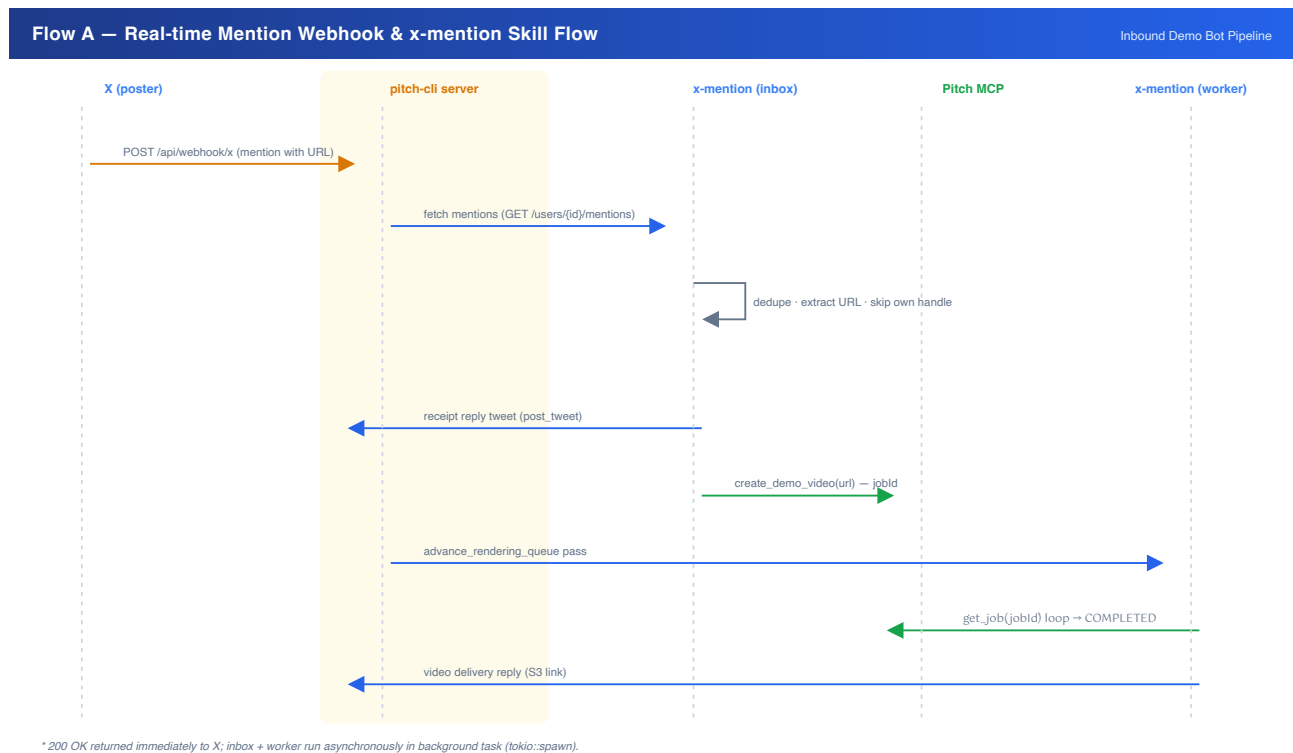


Figure 3: Sequence for the real-time mention webhook — `src/server.rs`, `src/inbox.rs`, `src/worker.rs`.

Blocking endpoint `POST /api/webhook/x` (X Account Activity API). The body is accepted and ignored; the handler returns immediately after spawning a background task. All heavy work happens in that task (`tokio::spawn`).

1. **CRC registration (GET).** `GET /api/webhook/x?crc_token=<t>` returns `sha256=<base64(HMAC-SHA256(t, X_CLIENT_SECRET))>` — `src/server.rs:93`.
2. **Receive mention (POST).** Spawn task runs `process_mention_inbox(false, false)` then `advance_rendering_queue(false, 10)` — `src/server.rs:136`.
3. **Fetch mentions.** `GET /users/{id}/mentions`, up to 20 — `src/x_api.rs:198`.
4. **Dedupe + filter.** Skip `@trypitchdotco` authors; skip tweets already present in `mention_jobs` (prevents double render / double reply) — `src/inbox.rs:46`.
5. **Extract URL.** Regex `https://...` first, then a bare domain — `src/inbox.rs:8`.
6. **No URL / s3.trypitch.co URL.** Record `status=no_url_found`, stop — `src/inbox.rs:63`.
7. **Receipt ack.** Post a thank-you reply (skipped under `--dry / --no-ack`) — `src/inbox.rs:80`.
8. **Submit render.** `create_demo_video(url)` via Pitch MCP; store `jobId` with `status=rendering` (or submitted if no `jobId` returned) — `src/inbox.rs:96`.
9. **Worker polls.** For each rendering/submitted job, call Pitch MCP `get_job` — `src/worker.rs:30`.
10. **Deliver.** On `COMPLETED`, extract the S3 artifact URL, post it as a reply in the original thread, mark delivered (`s3_video_url`, `x_reply_id` saved) — `src/worker.rs:50`.
11. **Fail response.** `FAILED/ERROR` status → mark failed — `src/worker.rs:85`.

4.2 Flow B — Manual trigger endpoint

`POST /api/webhook/trigger` accepts an optional JSON body `{"action": "..."}` and spawns a background pass (`src/server.rs:154`):

1. `action = growth` or `session` → first run `discover_prospects(5, false)`.
2. Always then run `process_mention_inbox(false, false) + advance_rendering_queue(false, 10)`.

Responds 200 {"status":"ok","action":...,"triggered":true} immediately. This is the hook an external scheduler or an operator curl uses for an on-demand pass.

4.3 Flow C — Outbound prospect discovery (x-prospect skill)

discover_prospects(max, dry) — src/discover.rs:67:

1. Run the fixed ICP query list against GET /tweets/search/recent — src/discover.rs:7.
2. Skip @trypitchdotco, @adnanspitch, and known handles — src/discover.rs:84.
3. Score the lead 1-10: +2 for a URL, +2 for competitor keywords (tella, screen studio, loom, guide, supademo, tango), +1 for intent words (need, looking for, alternative, how to, launched, building) — src/discover.rs:32.
4. Build a pitch hook and upsert the prospect (stage=new, segment=founder) into SQLite and state/prospects.jsonl — src/discover.rs:111.

4.4 Flow D — Health, stats & observability

1. GET /api/webhook/health → {"status":"ok",...} — src/server.rs:181.
2. GET /api/webhook/stats → mention_jobs_total, delivered, rendering, prospects_total — src/server.rs:194.
3. pitch-cli sync → the same summary on stdout — src/main.rs:378.
4. pitch-cli budget → per-action remaining caps + burst usage — src/safety.rs:157.
5. pitch-cli circuit-breaker → OK to run or PAUSED; exit code 1 when paused — src/safety.rs:50.

5 Webhook Routes Table

All routes are mounted twice: under `/api/webhook/...` and `/webhook/...` (`src/server.rs:71`). CORS is permissive; a trace layer logs every request.

Method	Path	Purpose
GET, /api/webhook/x, X-webhook-CRC challenge — HMAC-SHA256 token	POST, /api/webhook/x, X mention event → spawn inbox + worker pass	POST, /api/webhook/trigger, Manual/on-demand pass (<code>{"action":"mentions"\ "growth"}</code>)
GET, /api/webhook/health, Liveness probe	GET, /api/webhook/stats, SQLite pipeline counts	

Port note. Default is 8790 (`src/server.rs:52`), overridable via `-p/--port` or the `PORT` env var. Point a tunnel / reverse proxy at it so X (and your cron host) can reach it on a public URL.

6 Job & Prospect State Machines

Rows in `mention_jobs` are keyed on the unique `tweet_id` — the whole pipeline is idempotent, so overlapping webhook and cron passes never double-bill a render or double-post a reply.

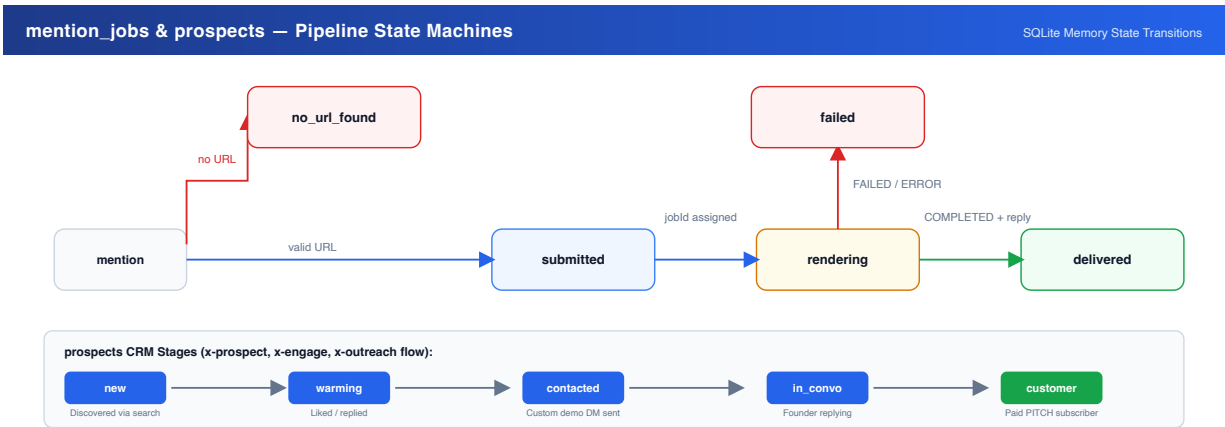


Figure 4: `mention_jobs` lifecycle — transitions driven by inbox and worker.

State	Meaning	Set by	— — —	no_url_found	Mention had no usable product URL	inbox	submitted
Render	submitted;	jobId not yet known	inbox	rendering	jobId assigned; render in progress	inbox / worker	delivered
Video	rendered;	S3 link replied in-thread	worker	failed	Pitch MCP reported FAILED/ ERROR	worker	

The worker status-polling loop can be dense (10 jobs per pass, configurable via `--limit`) — pair it with the cron cadence below so completed renders are delivered promptly.

7 Database Schema & State Mirroring

data/pitch_bot.db (SQLite, WAL), schema created at startup — src/db.rs:67:

Table	Role
mention_jobs, Idempotent render pipeline: tweet_id unique, status, editor_job_id, s3_video_url, x_reply_id	prospects, CRM pipeline: handle unique, score, stage, touches, next_action_date, product_url, notes, why; mirrored to state/prospects.jsonl
activities, Auditable action log (ts, action, handle, result); the input for budget; mirrored to state/activity-log.jsonl	insights, Adaptive memory blob (single row id=1) surfaced via db insights get

A few useful queries:

```
pitch-cli db jobs --status rendering // active renders
pitch-cli db jobs --status delivered // delivered demos
pitch-cli db prospects // CRM, newest first
pitch-cli db prospects --stage warming // warm cohort
pitch-cli db insights get // read adaptive memory
pitch-cli sync // pipeline summary
```

State mirror. upsert_prospect and log_activity also append/merge rows into .opencode/skills/x-growth/state/*.jsonl so the opencode skill suite sees the same CRM and activity trail (src/db.rs:403).

8 Schedules & Cron

The binary has **no internal scheduler**. Automation := the always-on server for real-time flow, plus a cron table driving the pass subcommands. The diagram shows the recommended operating cadence (times on a 24h ruler).

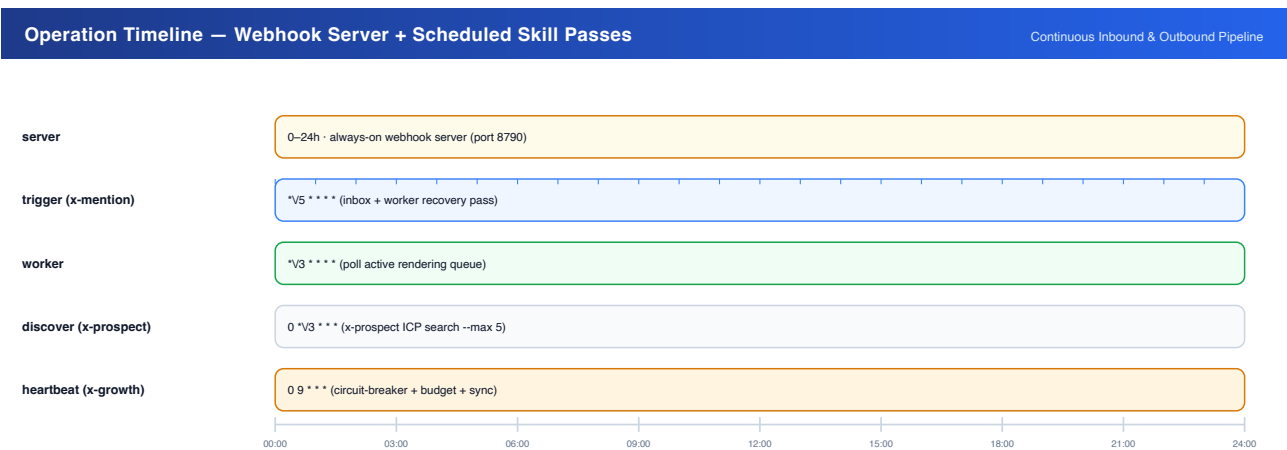


Figure 5: Recommended operations timeline — always-on server + polling cron.

8.1 The recovery net (why you still need cron)

X webhooks can be slow, batched, or occasionally dropped. The 5-minute trigger pass re-reads the same mentions (dedupe makes it safe), polls renders that the webhook-spawned worker had no time to finish, and advances submitted jobs. It is the difference between “webhook race” and “reliable pipeline”.

8.2 Server (always-on)

```
./target/release/pitch-cli server # 0.0.0.0:8790
# PORT=9000 ./target/release/pitch-cli server # override, or --port 9000
```

Run under a process manager (launchd / systemd / supervisor / tmux). It must be reachable from X via a public URL.

8.3 crontab

```
# every 5 min: ingest mentions + advance renders + DB summary (recovery net)
*/5 * * * * cd /opt/pitch-xbot && ./target/release/pitch-cli trigger >> /var/log/pitch/
trigger.log 2>&1

# optional narrower jobs instead of `trigger`:
# */10 * * * * ... pitch-cli inbox >> /var/log/pitch/inbox.log
# */3 * * * * ... pitch-cli worker >> /var/log/pitch/worker.log

# every 3 h during daylight: ICP prospective searches (rate-limited)
0 8-23/3 * * * cd /opt/pitch-xbot && ./target/release/pitch-cli discover --max 5 >> /var/
log/pitch/discover.log 2>&1

# daily heartbeat: breaker + budget + pipeline summary
0 9 * * * cd /opt/pitch-xbot && ./target/release/pitch-cli circuit-breaker && ./target/
release/pitch-cli budget && ./target/release/pitch-cli sync >> /var/log/pitch/daily.log
2>&1
```

8.4 Schedule summary

Pass	Cadence	Rationale
server, always-on, Real-time mention → demo flow	trigger, every 5 min, Recovery net; idempotent; advances render queue	discover, every 3 h, ICP discovery, spaced for rate limits
circuit-breaker + budget + sync, daily 09:00, Health + budget heartbeat	/api/webhook/ trigger, on demand, Arbitrary manual pass (mentions/growth)	

Burst hygiene. X anti-ban rules require ≥ 90 s between consecutive writes and a 1-hour burst ceiling (10/hr). The safety engine enforces this mechanically; cron pacing above must not burst beyond the per-action caps set in `src/safety.rs:177`.

9 Safety Engine

Every write pass starts by checking the **circuit breaker** then the **budget**. If the breaker is tripped or a cap is exhausted, the pass records nothing and stops — writes are skipped, never partially applied.

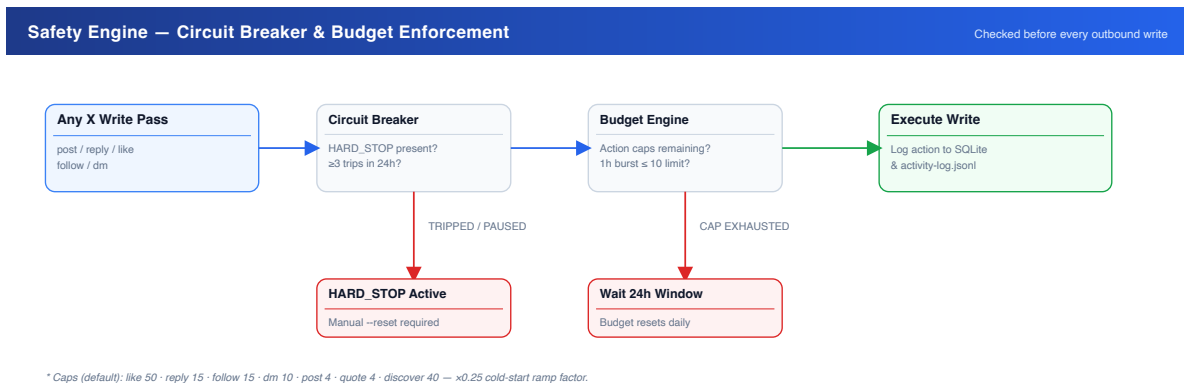


Figure 6: Gate for every X write — kill-switch + per-action budget.

9.1 Circuit breaker

1. File marker state/HARD_STOP present → **PAUSED** until --reset — src/safety.rs:50.
2. ≥3 trips within 24h → a HARD_STOP file is created automatically — src/safety.rs:105.
3. Manual trip: `pitch-cli circuit-breaker --trip "reason"` (appends to state/circuit-breaker.jsonl).
4. Resume: `pitch-cli circuit-breaker --reset`.

9.2 Budget caps (defaults, scaled ×0.25 during cold-start ramp)

Action	Daily cap
like	50
dm	10
reply	15
post	4
follow	15
quote	4
discover	40

`pitch-cli budget` prints used / remaining per action plus `actions_last_hour` and a `burst_warning` when ≥10 actions hit the trailing hour (src/safety.rs:157).

10 X API v2 & Pitch MCP

10.1 X API v2

XApiClient (src/x_api.rs) wraps the API base <https://api.twitter.com/2> and auto-refreshes the OAuth2 user token on a 401 — it rewrites X_USER_ACCESS_TOKEN / X_USER_REFRESH_TOKEN back into .env and retries once. Endpoints used: get_me, lookup_user, get_user_id, get_mentions, post_tweet (posts + replies), like_tweet, search_recent.

```
pitch-cli x-api me                # identity check
pitch-cli x-api refresh           # force a token refresh
pitch-cli x-api mentions          # recent incoming mentions
pitch-cli x-api reply 123 --text "... " # reply in-thread (--dry to preview)
pitch-cli x-api post --text "... "    # new tweet (--dry to preview)
pitch-cli x-api search "query"       # recent search
```

Known state (2026-08): the stored OAuth2 user tokens are expired/invalid — pitch-cli x-api me returns 401 even after auto-refresh. Fresh tokens must go into .env before any X API v2 write flow (receipts, delivery replies, discovery) will work. Legacy cache keys X_API_KEY, X_API_SECRET, X_BEARER_TOKEN are unused by the code.

10.2 Pitch MCP

JSON-RPC client to <https://api.trypitch.co/mcp> (src/pitch_mcp.rs), authorized via Authorization: Bearer \$PITCH_API_KEY, consuming the SSE text payload.

```
pitch-cli mcp credits              # confirm pro subscription + quota
pitch-cli mcp create <url> [instructions]
pitch-cli mcp status <job_id>
```

Render parameters are hardcoded defaults in create_demo_video (src/pitch_mcp.rs:62): voice Charon, light theme, ocean background, rounded shape, subtitles off.

11 Setup & Environment

11.1 Requirements

1. Rust 1.80+ (cargo / rustc)
2. A reachable public endpoint for the webhook server (or a tunnel)
3. data/pitch_bot.db directory writable by the running user

11.2 .env (gitignored — see src/config.rs)

```
X_CLIENT_ID=your_x_client_id
X_CLIENT_SECRET=your_x_client_secret
X_USER_ACCESS_TOKEN=your_oauth2_user_access_token
X_USER_REFRESH_TOKEN=your_oauth2_user_refresh_token
X_USER_ID=your_x_user_id
X_OPERATOR_HANDLE=@trypitchdotco
PITCH_API_KEY=your_pitch_api_key      # required; do NOT rely on the config.rs fallback
SQLITE_DB_PATH=./data/pitch_bot.db    # optional override
PORT=8790                             # optional override
```

Security: .env is gitignored and must never be committed. Do not log or print PITCH_API_KEY or the OAuth2 tokens. src/config.rs currently falls back to a hardcoded PITCH_API_KEY — treat that as a stopgap only.

11.3 Build & verify

```
cargo build --release                # → ./target/release/pitch-cli
./target/release/pitch-cli sync      # empty DB → zeros
./target/release/pitch-cli circuit-breaker # expect: OK to run (0 trips in 24h)
./target/release/pitch-cli budget    # confirm caps + 0 used
./target/release/pitch-cli mcp credits # confirm Pitch MCP auth
./target/release/pitch-cli server --port 8790 & # boot the server
curl http://localhost:8790/api/webhook/health # {"status":"ok",...}
curl "http://localhost:8790/api/webhook/x?crc_token=test" # sha256=...
```

11.4 Wire the webhook in X

1. In the X app, register https://<public-url>/api/webhook/x as the Account Activity webhook URL.
2. X calls GET (CRC) during registration and POST on every mention.
3. Confirm end-to-end: tweet @trypitchdotco with a product URL, watch /var/log/pitch/trigger.log, then check pitch-cli db jobs for the full lifecycle.

12 Automation Checklist

End-to-end bring-up and continuous operation:

1. .env populated with fresh OAuth2 user tokens + PITCH_API_KEY.
2. cargo build --release passes; binary on PATH (or absolute paths in cron).
3. pitch-cli circuit-breaker says OK to run; pitch-cli budget shows caps.
4. pitch-cli server always-on behind a public URL (health endpoint up).
5. Cron: */5 → pitch-cli trigger; 0 8-23/3 → pitch-cli discover --max 5.
6. Logs under /var/log/pitch/; watch GET /api/webhook/stats daily.
7. Daily heartbeat line fails loudly (exit code) when the breaker trips or caps hit zero — triage, then --reset after the window clears.

